

Statement Generation Portal — Deployment Guide

Solution: StatementPortal.sln • Branch: claude/batch-reversal-auth-optimize-67ewy3 • .NET 9 / ASP.NET Core Web API

This guide covers deploying the Statement Generation Portal backend: a resource-server API that validates bearer tokens issued by the existing LotusBank SSO service, enforces module-level RBAC, wraps the existing Statement Service for single/bulk statement generation, and maintains an append-only audit trail. It assumes the reader has access to the target host, the shared SingleSignOn database, and the Statement Service's service-account credentials.

Companion document: ARCHITECTURE.md in the repository root covers design rationale in depth. This guide focuses on the operational steps to get a build running in an environment.

1. Solution Overview

The solution has four projects, deployed as a single Web API (StatementPortal.Api references the other three):

Project	Responsibility
StatementPortal.Domain	Entities and enums only. No framework dependencies.
StatementPortal.Application	Use cases, validation (FluentValidation), interfaces. No EF Core or HTTP here.
StatementPortal.Infrastructure	EF Core, JWT validation, the Statement Service HTTP client, and the service-account authenticator for it.
StatementPortal.Api	Composition root: Program.cs, controllers, middleware, Swagger.

Authentication model: this service does not perform its own login. It validates the **CustomJwt** bearer token that LotusBank SSO already issued, using the same signing key SSO itself reads via `[dbo].[GetTokenSettings]`. There is no outbound network call to SSO at request time — this was a deliberate change after the previous design's call to SSO's `/auth-me` endpoint was found to fail intermittently with TLS handshake errors in a deployed environment (see Section 9, Troubleshooting).

Permissions are derived from the token's own `appRoles` claim, filtered to this application's entry only, by `AppRolesClaimsMiddleware`.

2. Prerequisites

2.1 Software

Component	Requirement
.NET SDK	.NET 9 SDK on the build machine; the ASP.NET Core 9 runtime on the host if not self-contained.
SQL Server	Access to the existing SingleSignOn database (read-only — <code>GetTokenSettings</code>), plus a database for this service's own audit trail (StatementPortal).
Seq	A reachable Seq server for structured logs. Existing shared instance, or a new one.
Reverse proxy / IIS	TLS termination in front of Kestrel — IIS with the ASP.NET Core Module, or nginx/Azure App Service/etc., matching how the rest of the LotusBank services are hosted.
Statement Service access	Network reachability to the existing Statement Service, plus a service-account username/password for it.

2.2 Access needed before you start

- A SQL login for the **StatementPortal** database with INSERT/SELECT-only rights on `AuditRecords` (see Section 5.2 — do not grant UPDATE/DELETE).
- Read access to the **SingleSignOn** database's `GetTokenSettings` stored procedure.
- The Statement Service's service-account username and password.

- Confirmation of this application's exact `ApplicationName` as registered in SSO's `appRoles` claim structure.
- The Seq server URL and, if this Seq instance enforces one, an API key.

3. Configuration Reference

Non-secret configuration lives in `appsettings.json` / `appsettings.<Environment>.json`. Anything marked **Secret** below must never be committed — use User Secrets locally, and environment variables or a secret store (Key Vault, etc.) in deployed environments.

Key	Secret?	Purpose
<code>ConnectionStrings:StatementPortal</code>	No*	This service's own database (audit trail). *The connection string itself isn't usually treated as a top-secret value, but avoid embedding a SQL-auth password in it — prefer Trusted/Managed Identity where possible.
<code>ConnectionStrings:SingleSignOnConnection</code>	No*	Same database SSO reads <code>GetTokenSettings</code> from.
<code>Authentication:ApplicationName</code>	No	Must exactly match this app's name in SSO's <code>appRoles</code> claim.
<code>StatementService:BaseUrl</code>	No	Base URL of the existing Statement Service.
<code>StatementService:AuthPath</code>	No	Relative path for the service-account login call.
<code>StatementService:Username</code>	Yes	Service-account username for the Statement Service.
<code>StatementService:Password</code>	Yes	Service-account password for the Statement Service.
<code>Seq:ServerUrl</code>	No	Seq ingestion endpoint.
<code>Seq:ApiKey</code>	Yes (if enforced)	Only needed if this Seq instance requires one.
<code>AllowedOrigins</code>	No	CORS allow-list — the frontend origin(s) permitted to call this API.
<code>KnownProxies</code>	No	IP address(es) of the trusted reverse proxy — required for correct X-Forwarded-For handling (see Section 8).

Why this matters operationally

`KnownProxies` must list the actual reverse proxy's IP. If it's left as the scaffold's placeholder value, or empty, client IP addresses recorded in the audit trail (US-10) will be wrong — every audit record will show the proxy's IP instead of the real caller's.

3.1 Setting secrets locally (development)

```
cd src/StatementPortal.Api
dotnet user-secrets set "StatementService:Username" "<value>"
dotnet user-secrets set "StatementService:Password" "<value>"
dotnet user-secrets set "Seq:ApiKey" "<value>"           # only if required
```

3.2 Setting secrets in a deployed environment

Use environment variables with the double-underscore separator ASP.NET Core's configuration system maps to nested keys, e.g. on a Windows/IIS host:

```
setx StatementService__Username "svc-statementportal"  
setx StatementService__Password "<value>"  
setx Seq__ApiKey "<value>"
```

On a container host, set the same names as container environment variables. Prefer a proper secret store (Key Vault / a managed secrets service) over plain environment variables where the platform supports it.

4. Database Setup

4.1 Apply migrations

Run this from a machine with network access to the target StatementPortal database, using the real connection string (via appsettings, an environment variable, or `--connection`):

```
dotnet tool install --global dotnet-ef # once, if not already installed

dotnet ef migrations add InitialCreate \
  -p src/StatementPortal.Infrastructure \
  -s src/StatementPortal.Api

dotnet ef database update \
  -p src/StatementPortal.Infrastructure \
  -s src/StatementPortal.Api
```

4.2 Lock down the audit table's permissions

After migrations run, apply this as a defense-in-depth measure so that even a future application defect cannot alter audit history — the application's own SQL login should not be able to UPDATE or DELETE from AuditRecords at all:

```
-- Run as an administrator, against the StatementPortal database
DENY UPDATE, DELETE ON dbo.AuditRecords TO [the_app_sql_login];
GRANT INSERT, SELECT ON dbo.AuditRecords TO [the_app_sql_login];
```

Do not skip this step

The application code itself has no Update/Delete method anywhere in its audit-writing path (by design), but this database-level grant is what protects the data even if that ever changes by mistake in a future release.

5. Build & Publish

```
dotnet restore StatementPortal.sln
dotnet build StatementPortal.sln -c Release

dotnet publish src/StatementPortal.Api -c Release -o ./publish
```

Copy the contents of `./publish` to the target host (or build directly on a CI runner and deploy the artifact via your normal pipeline).

6. Hosting (IIS example)

Matching how other LotusBank services in this ecosystem are hosted:

- 1 Install the **.NET 9 Hosting Bundle** on the IIS server if not already present, and restart IIS (`iisreset`) afterward.

- 2 Create (or reuse) an Application Pool set to **No Managed Code** — ASP.NET Core manages its own runtime.
 - 3 Create a Site or Virtual Application pointing at the published folder. If this service is mounted under a virtual path (e.g. /StatementPortalAPI), note that Swagger will then be reachable at <base>/StatementPortalAPI/swagger, not bare /swagger.
 - 4 Confirm the site's TLS binding uses a certificate issued for the correct hostname, with a complete chain (leaf + intermediates) — an incomplete chain is a common cause of downstream TLS handshake failures for callers of this API.
 - 5 Set the required environment variables / secrets on the Application Pool or via web.config's <environmentVariables> section, then recycle the app pool.
-

7. Post-Deployment Verification

- 1 **App started cleanly:** check the process/IIS logs for a clean startup with no unhandled exception — a missing required config value (e.g. Authentication:ApplicationName) throws at first use, not necessarily at startup, so also check the next steps.
- 2 **Swagger loads:** browse to <base-url>/swagger. You should see the API document with a green "Authorize" button (Bearer scheme).
- 3 **Unauthenticated request is rejected:** call GET /api/me with no token — expect 401, not a crash or a redirect to an HTML page.
- 4 **Authenticated request succeeds:** obtain a real token from SSO for a provisioned staff member, paste it into Swagger's Authorize dialog, and call GET /api/me — expect the staff's permissions list back.
- 5 **Permission enforcement works:** as a user without Audit permission, call GET /api/audit — expect 403.
- 6 **Statement Service connectivity:** submit a single statement request as a provisioned user and confirm it succeeds — this exercises the service-account login to the Statement Service end-to-end.
- 7 **Seq is receiving logs:** in Seq, filter by Application = 'StatementPortal' (or whatever this service is tagged as) and confirm the verification requests above appear, including the 401/403 cases from steps 3 and 5.

8. Logging & Monitoring

Every authentication failure (`OnAuthenticationFailed`, `OnChallenge`, `OnForbidden`) logs through `ILogger` with the request path, so 401/403s on any endpoint are visible in Seq without needing to reproduce the issue. Useful Seq queries once deployed:

```
Application = 'StatementPortal' and @Level = 'Warning'

Application = 'StatementPortal' and StatusCode in [401, 403]

Application = 'StatementPortal' and SourceContext = 'StatementPortal.Infrastructure.StatementService.State'
```

The `System.Net.Http.HttpClient` log category is set to `Debug` in `appsettings.json` specifically so outbound calls to the Statement Service surface request/response details in Seq if that integration ever fails — this was added after diagnosing a similar TLS issue with a different service in this ecosystem (see Section 9).

Note: the audit trail (Epics 6–8, stored in the `AuditRecords` table) is a separate, access-controlled compliance record — it is not written to Seq, and Seq is not a substitute for reviewing it via the Audit module's own API.

9. Troubleshooting

"The logger is already frozen" at startup

Caused by more than one `UseSerilog(...)/AddSerilog(...)` registration in the same process — Serilog's bootstrap-to-final logger handoff can only happen once. Search the solution for a second registration (a shared library's "standard logging setup" helper is the most common source) and remove it; only `Program.cs` should configure Serilog.

SSL/TLS handshake failures calling an external or internal service

If an outbound call from this host to another internal host fails with "The SSL connection could not be established":

- Confirm from the same host (not your workstation) with `curl -v https://the-target-host/` — this shows the real chain error.
- Most common cause on an internal `*.lotusbank.*` host: the internal CA certificate isn't installed in this host's trust store. Install it (`certutil -addstore -f "ROOT" internal-ca.cer` on Windows; `update-ca-certificates` on Linux/containers — Windows trust does not propagate into a Linux container automatically).
- Never resolve this by disabling certificate validation in code — that defeats TLS entirely for that call.

SQL Server error 40 ("Could not open a connection to SQL Server") while running migrations

The connection string points at a SQL Server instance that doesn't exist from wherever the command is running. Confirm whether the target is a default instance (`Server=.`), a named instance (`Server=.\SQLEXPRESS`), LocalDB (`Server=(localdb)\mssqllocaldb`), or a remote host requiring `Server=tcp:host,1433` with TCP/IP enabled and the firewall port open.

Swagger returns 404

- Confirm you're hitting /swagger, not the site root.
- If this service is mounted under a virtual application path on IIS, Swagger is at <virtual-path>/swagger.
- Confirm the deployed build is actually the current one — a stale build predating Swagger's addition will 404 regardless of source control state; redeploy and recycle the app pool.

10. Pre-Go-Live Security Checklist

- All Secret-marked config values (Section 3) are set via environment variable / secret store — none committed to `appsettings.json`.
 - `AuditRecords` table has the INSERT/SELECT-only grant applied (Section 4.2).
 - `KnownProxies` is set to the real reverse proxy IP(s), not a placeholder.
 - `AllowedOrigins` lists only the actual frontend origin(s) — no wildcard.
 - TLS certificate on the public/internal binding is valid, not expiring imminently, and includes the full chain.
 - `Authentication:ApplicationName` matches this app's registration in SSO exactly (case as SSO expects it).
 - Verification steps in Section 7 all pass, including the negative cases (401/403).
 - Seq is receiving this service's logs and someone is watching for repeated 401/403/500 patterns post-launch.
-

11. Rollback

This service is stateless (no session store of its own — see ARCHITECTURE.md Section 3), so rollback is a standard redeploy of the previous build artifact/container image. The only stateful piece is the `StatementPortal` database: if a rollback needs to undo a migration, use:

```
dotnet ef database update <PreviousMigrationName> \  
-p src/StatementPortal.Infrastructure \  
-s src/StatementPortal.Api
```

Never roll back past a migration that has already written real audit records the schema change depends on — check with the team before reverting a migration in a live environment.